

Image Router Studiovision Monitor

Rapport Technique — Documentation du Projet

Versions V1 et V2 — Analyse Comparative

Domaine	Automatisation d'imagerie médicale
Technologies	Python 3, watchdog, pywin32, pyodbc
Environnement	Windows 7 / Windows 10/11, MS Access
Date	Mai 2026

Table des matières

1	Introduction et Objectifs	2
1.1	Contexte clinique	2
1.2	Objectifs du projet	2
1.3	Périmètre de déploiement	2
2	Configuration et Architecture des Dossiers	2
2.1	Variables de configuration centralisées	2
2.2	Rôle de chaque chemin	3
2.3	Structure hiérarchique des dossiers patients	4
3	Architecture Technique et Flux de Données	4
3.1	Vue d'ensemble du flux	4
3.2	Surveillance en temps réel avec <code>watchdog</code>	4
3.3	Gestion de la concurrence : Queue et thread Worker	5
3.4	Gestion des verrous de fichiers	5
3.5	Logging	5
4	Intégration Microsoft Access — Le Cœur du Système	6
4.1	Lecture du contexte patient via COM	6
4.2	Résolution du dossier patient via ODBC	7
4.3	Insertion du nouveau document dans la base	7
5	Évolution et Optimisations : De la V1 à la V2	8
5.1	Vue d'ensemble des versions	8
5.2	Optimisation du rafraîchissement UI : mécanisme de Debounce	8
5.2.1	Comportement en V1 — Rafraîchissement immédiat (problématique)	8
5.2.2	Comportement en V2 — Rafraîchissement groupé (Burst Debounce)	9
5.3	Correction de régression UI : ciblage précis du sous-formulaire SFDoc	10
5.3.1	Problème en V1 — Regression de navigation	10
5.3.2	Correction en V2 — Fonction <code>_find_sfdoc()</code>	10
5.4	Synthèse des améliorations V1 → V2	11
6	Traitement Spécifique : Équipements Nidek (Box 2)	11
6.1	Particularités du format de sortie Nidek	11
6.2	Pipeline de traitement Nidek	12
6.3	Différence V1/V2 pour Box 2	13
6.4	Matrice de déploiement par poste	13
7	Conclusion	13

1. Introduction et Objectifs

1.1 Contexte clinique

Dans un environnement ophtalmologique, chaque examen d'imagerie médicale — qu'il s'agisse d'un rétinographe, d'un OCT (Optical Coherence Tomography), d'un appareil Nidek ou de tout autre équipement — produit des fichiers numériques (images JPEG, TIFF, BMP, DICOM, etc.) qui doivent être associés au bon dossier patient dans le système de gestion médicale **Studiovision** (basé sur Microsoft Access).

Cette association, si elle est effectuée manuellement, représente une source d'erreurs critiques (fichier rattaché au mauvais patient) et une perte de temps significative pour le personnel médical. La nécessité d'un processus d'acheminement automatique, fiable et transparent s'est donc imposée.

1.2 Objectifs du projet

Le projet **Image Router** (rebaptisé **Studiovision Monitor**) a pour objectifs de :

- Surveiller en temps réel un ou plusieurs répertoires sources afin de détecter l'apparition de nouveaux fichiers d'imagerie médicale.
- Identifier automatiquement le patient en cours de consultation en interrogeant l'interface active de l'application Microsoft Access via le protocole COM (Component Object Model).
- Déplacer le fichier détecté vers le dossier réseau dédié au patient concerné.
- Enregistrer le document dans la base de données Studiovision (PUBLIC.MDB) via une connexion ODBC.
- Rafraîchir l'interface utilisateur d'Access pour que le nouveau document soit immédiatement visible dans le sous-formulaire de gestion des documents.
- Gérer les cas d'erreur de manière non-bloquante : fichiers non rattachables (*orphelins*), verrous de fichiers persistants, panne COM, etc.

1.3 Périmètre de déploiement

Le service est déployé sur plusieurs postes de travail Windows (*Box 1, Box 2, Box Pierre Henris, Box 6*), chacun disposant de sa propre configuration de chemins. Le parc inclut des machines sous Windows 7 (nécessitant une variante de compatibilité du code) et des machines sous Windows 10/11.

2. Configuration et Architecture des Dossiers

2.1 Variables de configuration centralisées

Toute la configuration du service est regroupée en tête de chaque script sous la forme de constantes Python, ce qui facilite l'adaptation du service à chaque poste sans modifier la logique applicative.

```
1 SOURCE_DIR = Path(r"??") # Dossier surveillé par watchdog
2 ORPHAN_DIR = Path(r"??") # Dossier de quarantaine
```

```

3 DEST_PHOTOS = Path(r"??") # Racine des photos patients sur le
   reseau
4 PUBLIC_MDB   = Path(r"??") # Base de donnees principale (lecture/
   ecriture)
5 DOCUM_MDB    = Path(r"??") # Base secondaire (consultation
   uniquement)

```

Listing 1 – Variables de configuration principales (template)

2.2 Rôle de chaque chemin

SOURCE_DIR

Répertoire surveillé en temps réel par l'observateur `watchdog`. C'est dans ce dossier (ou ses sous-dossiers pour les équipements Nidek) que les appareils d'imagerie médicale déposent leurs fichiers. La surveillance est récursive (`recursive=True`).

Exemples par poste :

- Box 2 : \\nidek-box\master\ (réseau Nidek)
- Box 1 : C:\Box 1 (dossier local d'export CV)
- Box Preconsulte 2 : C:\RTOCTVIEW\images OCT et rétino
- Box 6 : \\studiovision\Studiov2000\BIOMETRIES

DEST_PHOTOS

Racine du répertoire réseau partagé contenant les photos patients, organisées en sous-dossiers de groupes et de patients. Le service construit le chemin complet de destination en combinant cette racine avec les informations extraites de la base de données.

Valeur commune : \\studiovision\Studiov2000-OM\PHOTOS

ORPHAN_DIR

Dossier de quarantaine local où sont déplacés les fichiers qui n'ont pu être rattachés à aucun patient (absence de patient actif après timeout, échec de résolution du dossier, etc.). Ce mécanisme garantit qu'aucun fichier médical n'est perdu, même en cas d'erreur.

Exemples : C:\Users\Admin\Desktop\Images_Oubliées

PUBLIC_MDB

Base de données Microsoft Access principale du système Studiovision. C'est dans cette base que se trouvent la table `Documents` (lecture pour résolution du dossier patient, écriture pour insertion du nouveau document) et les informations de patients.

Chemin : \\studiovision\Studiov2000-OM\fichier\PUBLIC.MDB

DOCUM_MDB

Base de données secondaire, référencée en configuration mais utilisée en lecture seule dans le périmètre actuel du service. Les insertions sont exclusivement réalisées dans `PUBLIC.MDB`.

Chemin : \\studiovision\Studiov2000-OM\fichier\DOCUM.MDB

2.3 Structure hiérarchique des dossiers patients

La résolution du dossier de destination d'un patient suit une logique de chemin relatif encodé dans la base de données. Le champ [Photo externe] de la table Documents contient un chemin de la forme :

\GROUPE\PATIENT_ID\nom_fichier.ext

Le service extrait les deux premiers niveaux (GROUPE et PATIENT_ID) pour reconstruire le chemin absolu :

```
1 parts = row[0].strip().strip("\\").split("\\")
2 folder = DEST_PHOTOS / parts[0] / parts[1]
3 # Resultat : \\studiovision\Studiov2000-OM\PHOTOS\GROUPE\PATIENT_ID
```

Listing 2 – Reconstruction du chemin destination

3. Architecture Technique et Flux de Données

3.1 Vue d'ensemble du flux

Le service repose sur une architecture producteur/consommateur à un seul thread de traitement, conçue pour éviter tout blocage de l'interface utilisateur Access et toute perte de fichier.



3.2 Surveillance en temps réel avec watchdog

La bibliothèque `watchdog` est utilisée pour s'abonner aux événements du système de fichiers Windows (`ReadDirectoryChangesW`) sans polling actif. La classe `ImageProducer` hérite de `FileSystemEventHandler` et réagit uniquement à l'événement `on_created`, filtrant les fichiers selon la liste d'extensions autorisées :

```
1 WATCHED_EXTENSIONS = {
2     ".jpg", ".jpeg", ".jif", ".png",
3     ".bmp", ".tif", ".tiff", ".dcm"
4 }
5
6 class ImageProducer(FileSystemEventHandler):
7     def on_created(self, event) -> None:
8         if event.is_directory:
9             return
10        file = Path(event.src_path)
11        if file.suffix.lower() not in WATCHED_EXTENSIONS:
12            return
13        self._queue.put(file)
```

Listing 3 – Gestionnaire d'événements watchdog

L'observateur est démarré en mode récursif, ce qui permet de surveiller les sous-dossiers créés dynamiquement par les équipements (notamment les équipements Nidek).

3.3 Gestion de la concurrence : Queue et thread Worker

Un objet `queue.Queue` est utilisé comme canal de communication thread-safe entre le producteur (thread principal/watchdog) et le consommateur (thread Worker). Ce choix architectural présente plusieurs avantages :

- Les opérations longues (attente de verrou, interrogation COM, accès réseau) sont entièrement confinées dans le thread Worker, sans jamais bloquer le thread watchdog.
- En cas d'arrivée simultanée de plusieurs fichiers (rafale d'images), ils sont sérialisés dans la file et traités séquentiellement, évitant toute condition de course sur les ressources partagées (base de données, interface Access).
- L'arrêt propre est garanti par `file_queue.join()`, qui attend l'épuisement de la file avant de terminer le processus.

```

1 file_queue = queue.Queue()
2 worker_thread = threading.Thread(
3     target=worker, args=(file_queue,), name="Worker", daemon=True
4 )
5 worker_thread.start()
6
7 observer = Observer()
8 observer.schedule(producer, str(SOURCE_DIR), recursive=True)
9 observer.start()

```

Listing 4 – Démarrage du Worker et de l'observateur

3.4 Gestion des verrous de fichiers

Les appareils d'imagerie écrivent leurs fichiers en plusieurs étapes. Il est fréquent qu'un fichier soit détecté par watchdog alors qu'il est encore en cours d'écriture et verrouillé par le processus émetteur. Le service implémente un mécanisme de retry avec les paramètres suivants :

Paramètre	Valeur	Signification
FILE_LOCK_RETRY_DELAY	3 s	Pause entre chaque tentative
FILE_LOCK_MAX_ATTEMPTS	15	Nombre maximal de tentatives
<i>Attente maximale totale</i>		<i>45 secondes</i>

Si le fichier reste verrouillé après 15 tentatives, il est abandonné avec une erreur critique dans les logs. Si aucun patient actif n'est trouvé dans Access après `PATIENT_WAIT_TIMEOUT` = 900 secondes (15 minutes), le fichier est déplacé vers `ORPHAN_DIR`.

3.5 Logging

Le service journalise ses opérations simultanément vers la console et un fichier `image_router.log`, avec horodatage et nom du thread :

```

1 # Format: "2026-05-06 09:52:10 INFO [Worker] Message"
2 logging.basicConfig(
3     level=logging.INFO,
4     format="%(asctime)s %(levelname)-8s [%(threadName)s] %(
5 message)s",
6     handlers=[
7         logging.FileHandler("image_router.log", encoding="utf-8"),
8         logging.StreamHandler(sys.stdout),
9     ],
10 )

```

Listing 5 – Format de log

4. Intégration Microsoft Access — Le Cœur du Système

4.1 Lecture du contexte patient via COM

L'interaction avec Microsoft Access repose sur la bibliothèque `pywin32` (`win32com.client`), qui permet d'obtenir une référence à l'instance Access en cours d'exécution via le protocole COM :

```

1 def get_active_patient() -> dict | None:
2     access = win32com.client.GetObject("Access.Application")
3     form = access.Screen.ActiveForm
4     if form is None:
5         return None
6
7     target = {"Code patient", "NOM", "Prenom"}
8     data = {}
9
10    for i in range(form.Controls.Count):
11        ctrl = form.Controls(i)
12        try:
13            if str(ctrl.Name) in target:
14                data[ctrl.Name] = ctrl.Value
15        except Exception:
16            pass
17
18    return {
19        "code": str(data["Code patient"]),
20        "nom": str(data["NOM"]),
21        "prenom": str(data["Prenom"]),
22    }

```

Listing 6 – Récupération du patient actif via COM

Note technique : `GetObject("Access.Application")` retourne une référence COM à l'instance Access déjà ouverte sur le poste. Cette approche est non-invasive : elle lit les valeurs des contrôles du formulaire actif sans modifier l'état de l'application. Toutes les erreurs COM sont capturées et loggées au niveau DEBUG pour éviter les faux positifs dans les logs opérationnels.

Initialisation COM obligatoire : Le thread Worker appelle `pythoncom.CoInitialize()` en début d'exécution et `pythoncom.CoUninitialize()` dans le bloc `finally`. Cette initialisation est indispensable pour tout accès COM depuis un thread secondaire sous Windows.

Les champs attendus dans le formulaire Access sont configurables :

```
1 ACCESS_FIELD_CODE    = "Code patient"
2 ACCESS_FIELD_NOM     = "NOM"
3 ACCESS_FIELD_PRENOM  = "Prenom"
```

Listing 7 – Champs Access configurables

4.2 Résolution du dossier patient via ODBC

Une fois le code patient récupéré depuis l'interface Access, le service interroge la table `Documents` de `PUBLIC.MDB` pour retrouver un enregistrement existant pour ce patient et en déduire le chemin de son dossier de photos :

```
1 def find_patient_folder(patient_code: str) -> Path | None:
2     conn = db_connect(PUBLIC_MDB)
3     cursor = conn.cursor()
4     cursor.execute(
5         "SELECT TOP 1 [Photo externe] FROM Documents "
6         "WHERE [code patient] = ? AND [Photo externe] IS NOT NULL",
7         (int(patient_code),)
8     )
9     row = cursor.fetchone()
10    conn.close()
11
12    parts = row[0].strip().strip("\\").split("\\")
13    folder = DEST_PHOTOS / parts[0] / parts[1]
14    return folder if folder.is_dir() else None
```

Listing 8 – Résolution du dossier patient

La connexion ODBC est construite avec le pilote natif Access :

```
1 def db_connect(mdb_path: Path):
2     return pyodbc.connect(
3         f"DRIVER={{Microsoft Access Driver (*.mdb, *.acldb)}};"
4         f";DBQ={mdb_path};"
5     )
```

Listing 9 – Chaîne de connexion ODBC

4.3 Insertion du nouveau document dans la base

Après déplacement du fichier vers le dossier patient, un enregistrement est inséré dans la table `Documents` de `PUBLIC.MDB`. Le champ `TypeVW` est fixé à 99 (type document externe), et `NumDocExterne` est laissé à `NULL`.

```
1 cursor.execute("""
2     INSERT INTO Documents
3         ([code patient], [Date], DESCRIPTIONS, TEXTE,
4         [Photo externe], TypeVW, NumDocExterne)
```



```

5     VALUES (?, ?, ?, ?, ?, 99, NULL)
6     """
7     (int(patient["code"]), datetime.now(),
8      description, relative_path, relative_path)
9     )
10    conn.commit()

```

Listing 10 – Insertion SQL du nouveau document

Le champ DESCRIPTIONS reçoit une valeur dépendant du type de fichier traité :

Extension(s)	Valeur de DESCRIPTIONS
.jpg, .jpeg, .jfif, .png, .bmp	Image
.tif, .tiff	OCT
.dcm	DICOM

5. Évolution et Optimisations : De la V1 à la V2

5.1 Vue d'ensemble des versions

Script	Cible	Version
studiovision_monitor.py	Box Pierre Henris	V1
studiovision_monitorV2.py	Box 1	V2
windows7.py	Postes Windows 7	V1 (compat. Py3.8)
windows7V2.py	Postes Windows 7	V2 (compat. Py3.8)
box2.py	Box 2 (Nidek)	V1 + Nidek
box2V2.py	Box 2 (Nidek)	V2 + Nidek

Compatibilité Windows 7 : Les variantes windows7.py et windows7V2.py utilisent la syntaxe `Optional[dict]` du module `typing` au lieu de la notation `dict | None` (PEP 604), incompatible avec Python 3.8 (dernier interpréteur disponible sous Windows 7). Le formatage des chaînes de log utilise `%s` au lieu des f-strings pour la même raison.

5.2 Optimisation du rafraîchissement UI : mécanisme de De-bounce

5.2.1 Comportement en V1 — Rafraîchissement immédiat (problématique)

En V1, après chaque insertion réussie dans la base de données, le service attend 1,5 seconde puis déclenche immédiatement un rafraîchissement de l'interface Access :

```

1  if insert_document(patient, relative_path, description):
2      # Rafraîchissement immédiat apres chaque fichier
3      time.sleep(1.5)
4      refresh_ui()

```

Listing 11 – V1 : rafraîchissement synchrone à chaque fichier

Problème : Lorsqu'un équipement exporte plusieurs images en rafale (exemple : rétinographe produisant 5 à 10 images pour un même examen), Access reçoit autant

d'appels COM `Requery()` successifs. Chaque appel provoque un blocage momentané de l'interface utilisateur (gel du formulaire pendant la requête) et un repositionnement potentiellement indésirable du curseur d'enregistrement.

5.2.2 Comportement en V2 — Rafraîchissement groupé (Burst Debounce)

La V2 introduit un mécanisme de *debounce* basé sur un drapeau booléen `needs_refresh` et un timeout de 1,5 s sur le `queue.get()` :

```

1  needs_refresh = False
2
3  while True:
4      try:
5          # Attente avec timeout : si la file est vide pendant 1.5 s,
6          # on considere que la rafale est terminee
7          file = file_queue.get(timeout=1.5)
8      except queue.Empty:
9          if needs_refresh:
10             log.info("Burst complete -- triggering batched UI
refresh.")
11             refresh_ui()
12             needs_refresh = False
13             continue
14
15     # ... traitement du fichier ...
16
17     if insert_document(patient, relative_path, description):
18         needs_refresh = True # Lever le drapeau, NE PAS rafraichir
maintenant

```

Listing 12 – V2 : logique de debounce dans le Worker

Mécanisme : Le thread Worker attend un fichier en bloquant sur `queue.get(timeout=1.5)`. Tant que des fichiers arrivent à intervalles inférieurs à 1,5 s, le Worker les traite en séquence et lève simplement le drapeau `needs_refresh`. Dès que la file reste vide pendant 1,5 secondes consécutives (fin de rafale), un unique appel `refresh_ui()` est émis.

	V1	V2
Rafraîchissements pour 8 images	8 appels COM	1 appel COM
Gel potentiel d'Access	À chaque image	À la fin de la rafale
Délai visible pour l'utilisateur	$8 \times 1.5 \text{ s} = 12 \text{ s}$	1.5 s

Un flush de sécurité est également prévu à l'arrêt du Worker :

```

1  finally:
2      if needs_refresh:
3          log.info("Worker shutting down -- flushing pending UI
refresh.")
4          refresh_ui()
5          pythoncom.CoUninitialize()

```

Listing 13 – V2 : flush du refresh lors de l'arrêt

5.3 Correction de régression UI : ciblage précis du sous-formulaire SFDoc

5.3.1 Problème en V1 — Regression de navigation

En V1, la fonction `refresh_ui()` appelle `_requery_form(form)` qui requiert récursivement *tous* les formulaires et sous-formulaires en commençant par les enfants, puis le formulaire parent :

```

1 def _requery_form(form) -> None:
2     # Requery de tous les sous-formulaires enfants
3     for i in range(form.Controls.Count):
4         ctrl = form.Controls(i)
5         try:
6             if ctrl.ControlType == _AC_SUBFORM:
7                 _requery_form(ctrl.Form)
8         except Exception:
9             pass
10    # Puis requery du formulaire courant
11    form.Requery()
12
13 def _goto_last_record(form) -> None:
14    # Cherche SFDoc et appelle MoveLast()
15    for i in range(form.Controls.Count):
16        ctrl = form.Controls(i)
17        if ctrl.Name == SFD doc _SUBFORM_NAME:
18            ctrl.Form.Recordset.MoveLast()

```

Listing 14 – V1 : requery récursif du formulaire entier

Effets de bord : Le `Requery()` du formulaire parent réinitialise le `Recordset` de ce dernier, renvoyant l'utilisateur à l'enregistrement #1 de la liste des patients — comportement extrêmement gênant en consultation, où le praticien travaille sur un patient précis.

5.3.2 Correction en V2 — Fonction `_find_sfdoc()`

La V2 introduit la fonction `_find_sfdoc(form)` qui parcourt l'arbre des contrôles pour localiser *uniquement* le sous-formulaire SFDoc, sans jamais toucher au formulaire parent :

```

1 def _find_sfdoc(form):
2     """
3     Parcourt récursivement l'arbre des controles et retourne
4     l'objet Form du sous-formulaire nommé SFD doc _SUBFORM_NAME.
5     Ne touche pas au Recordset du formulaire parent.
6     """
7     for i in range(form.Controls.Count):
8         ctrl = form.Controls(i)
9         try:
10            if ctrl.ControlType != _AC_SUBFORM:
11                continue
12            if ctrl.Name == SFD doc _SUBFORM_NAME:
13                return ctrl.Form # Retourne le Form du sous-
14            formulaire
15            found = _find_sfdoc(ctrl.Form)
16            if found is not None:

```

```

16         return found
17     except Exception:
18         pass
19     return None
20
21 def refresh_ui() -> None:
22     access = win32com.client.GetObject("Access.Application")
23     form    = access.Screen.ActiveForm
24
25     sfdoc = _find_sfdoc(form)
26     if sfdoc is None:
27         log.warning("SFDoc non trouve. Refresh ignore.")
28         return
29
30     sfdoc.Requery()    # Seul SFDoc est requeried
31     sfdoc.Recordset.MoveLast() # Navigation vers le dernier
    document

```

Listing 15 – V2 : recherche ciblée du sous-formulaire SFDoc

Impact : En V2, le Recordset du formulaire parent (liste des patients) n'est jamais modifié. L'utilisateur reste positionné sur le patient en cours de consultation. Seul le sous-formulaire SFDoc est actualisé et positionné sur le dernier document ajouté.

5.4 Synthèse des améliorations V1 → V2

Aspect	V1	V2
Déclenchement du refresh	Immédiat, après chaque fichier	Différé, à la fin de chaque rafale
Portée du Requery()	Formulaire entier + sous-formulaires	Sous-formulaire SFDoc uniquement
Risque de gel d'Access	Élevé (appels COM répétés)	Minimal (un seul appel par rafale)
Régression de navigation	Oui (renvoi au patient #1)	Non (curseur parent préservé)
Compatibilité Python 3.8	Non (dict None)	Oui (Optional[dict]) pour les variantes Win7

6. Traitement Spécifique : Équipements Nidek (Box 2)

6.1 Particularités du format de sortie Nidek

L'équipement **Nidek** (scanner réseau, accessible via \\nidek-box\master\) adopte une organisation de fichiers hiérarchique spécifique. Pour chaque acquisition, il crée :

- Un **dossier principal** (main_dir) identifiant l'acquisition.
- Un **sous-dossier de scan** (scan_dir) contenant les fichiers.
- Plusieurs **fichiers images** : une image principale haute résolution et une ou plusieurs images miniatures (thumbnails).

— Des **fichiers XML** de métadonnées résiduels, sans utilité pour Studiovision.

L'organisation est détectée automatiquement par la profondeur du chemin :

```

1 scan_dir = file.parent
2 main_dir = file.parent.parent
3 is_nidek = main_dir.parent == SOURCE_DIR
4 # Un fichier Nidek est a 2 niveaux sous SOURCE_DIR

```

Listing 16 – Détection des fichiers Nidek

6.2 Pipeline de traitement Nidek

Le traitement Nidek se déroule en plusieurs étapes séquentielles :

1. **Vérification des fichiers résiduels** : si le dossier de scan a déjà été traité (il figure dans l'ensemble `processed_scan_dirs`), le fichier courant est un résidu et est supprimé silencieusement.
2. **Stabilisation de l'écriture** : une pause de 2 secondes est observée pour laisser l'équipement terminer l'écriture de tous les fichiers du scan.
3. **Nettoyage des fichiers XML** : tous les fichiers `*.xml` présents dans le dossier de scan sont supprimés.

```

1 for xml_file in list(scan_dir.glob("*.xml")):
2     xml_file.unlink()
3     log.info(f"[NIDEK] XML removed: {xml_file.name}")

```

Listing 17 – Suppression des fichiers XML Nidek

4. **Identification de l'image principale** : parmi tous les fichiers images du dossier de scan, le fichier de plus grande taille (`st_size`) est retenu comme image principale. Les autres (miniatures) sont supprimés.

```

1 sibling_images = [
2     f for f in scan_dir.iterdir()
3     if f.is_file() and f.suffix.lower() in WATCHED_EXTENSIONS
4 ]
5 largest_image = max(sibling_images, key=lambda f: f.stat().
6     st_size)
7 if file.resolve() != largest_image.resolve():
8     file.unlink() # Suppression du thumbnail
9     file_queue.task_done()
10    continue

```

Listing 18 – Sélection de l'image principale par taille

5. **Traitement standard** : l'image principale est soumise au flux classique (lookup patient COM, déplacement, insertion DB).
6. **Nettoyage des dossiers sources** : les dossiers `scan_dir` et `main_dir` sont supprimés s'ils sont vides après le déplacement.

```

1 def _try_rmdir(folder: Path) -> None:
2     if folder.is_dir() and not any(folder.iterdir()):
3         folder.rmdir()

```

Listing 19 – Nettoyage des dossiers Nidek vides

6.3 Différence V1/V2 pour Box 2

En V2 (`box2V2.py`), le nettoyage des dossiers est déplacé *après* l'insertion en base de données (et non après le déplacement du fichier comme en V1), garantissant que les dossiers ne sont jamais supprimés avant que la transaction DB soit confirmée. Le mécanisme de debounce s'applique également aux fichiers Nidek.

6.4 Matrice de déploiement par poste

Poste	Script	OS	Particularité
Box 1	<code>studiovision_monitorV2.py</code>	Win 10/11	Source : dossier local Export CV
Box 2	<code>box2V2.py</code>	Win 10/11	Source : réseau Nidek, traitement spécifique
Box 3	—	—	Non déployé
Box 4	<i>(config à définir)</i>	Win 10/11	Chemin SOURCE_DIR non encore renseigné
Box 5	—	—	Matériel insuffisant (trop peu puissant)
Box 6	<code>box2.py</code> ou similaire	Win 10/11	Source : BIOMETRIES sur réseau
Box Pierre Henris	<code>studiovision_monitor.py</code>	Win 10/11	V1, source locale

Action requise pour Box 4 : Le chemin SOURCE_DIR de Box 4 doit être défini avant déploiement. Tous les autres paramètres (chemins réseau Studiovision, bases MDB) sont identiques aux autres postes.

7. Conclusion

Le projet **Image Router / Studiovision Monitor** constitue une solution robuste d'automatisation de l'acheminement des images médicales, reposant sur un empilement technologique maîtrisé (Python, watchdog, pywin32, pyodbc) et parfaitement adapté aux contraintes de l'environnement clinique existant.

L'évolution de la V1 vers la V2 a apporté deux améliorations majeures et mesurables :

1. L'élimination des gels répétés de l'interface Access lors des rafales d'images, grâce au mécanisme de debounce sur la file de traitement.
2. La correction d'un bug de navigation critique qui renvoyait l'utilisateur au premier enregistrement de la liste patients à chaque rafraîchissement, par le ciblage précis du seul sous-formulaire SFDoc.

Le traitement spécifique des équipements Nidek (Box 2) démontre la capacité d'extension du système à de nouvelles sources sans modification de l'architecture core.

Les prochaines étapes identifiées sont : la configuration de `SOURCE_DIR` pour Box 4 et la vérification de la compatibilité du service sur ce poste.